

Proyecto 2 - Parte 3: Pruebas de rendimiento de Qolqe Wallet

Curso: CS5383 – Pruebas y Verificación de Software · **Peso:** 20 % del Proyecto 2

1. Objetivo

Diseñar, ejecutar y analizar pruebas de rendimiento de Qolqe Wallet a partir de un requerimiento de negocio, y documentarlas en el **Informe de Pruebas de Rendimiento**, usando la plantilla adjunta `Template_Informe_Rendimiento.docx`.

Todos los grupos trabajan con **el mismo requerimiento** (sección 4).

2. Herramienta y entorno

- **Herramienta:** de libre elección (k6, JMeter, Gatling, Locust, Artillery, Taurus u otra).
- **Si usan JMeter:** diseñen el plan en la interfaz gráfica, pero **ejecútenlo por consola**:
  `jmeter -n -t plan.jmx -l resultados.jtl -e -o reporte/`
- **Entorno:** elijan una de estas opciones y documéntenla en el informe:

Opción	Descripción
Local	Aplicación y herramienta en la misma laptop
Dos equipos	Una laptop ejecuta Qolqe y la otra genera la carga, en la misma red
Docker	Qolqe en un contenedor con CPU y memoria limitadas (por ejemplo, `--cpus=1 --memory=1g`)
GitHub Codespaces	Qolqe en un Codespace y la carga desde la laptop, por el puerto público

3. Qué deben hacer

1. **Identificar los flujos críticos** del requerimiento (mínimo tres) y justificar el criterio: frecuencia, impacto en el dinero del usuario, riesgo.
2. **Convertir el requerimiento en umbrales** (métrica + percentil + valor límite + carga). Si una parte no se puede convertir tal como está escrita, indiquen qué falta y el **supuesto** que toman.
3. **Diseñar el modelo de carga** (VUs, ramp-up, duración, *think time*, distribución de operaciones). Cada valor debe salir de un dato del requerimiento.
4. **Preparar datos de prueba**: un usuario distinto por VU, con saldo suficiente.
5. **Ejecutar** al menos una prueba de **carga** y una de **estrés**. Pico y resistencia: ejecútenlas o expliquen por qué no aplican.
6. **Medir CPU y memoria** de la aplicación durante cada ejecución.
7. **Analizar**: ¿pasó? → ¿dónde falló? → ¿por qué? (cuello de botella con evidencia) → ¿qué hacer?
8. **Completar el informe** y **exponer** con demostración en vivo.

4. Requerimiento de rendimiento

Preparación para la campaña "Quincena Qolqe"

Antes de la campaña necesitamos saber si la plataforma soporta la demanda esperada y qué tan lejos está de su límite.

Uso actual. En la **hora punta** (12:00 a 13:00) se inician unas **3 000 sesiones por hora**. Una sesión dura en promedio **2 minutos**: el usuario inicia sesión y realiza **6 operaciones**, con una pausa de **10 a 25 segundos** (18 s en promedio) entre una y otra. Las operaciones se reparten como indica la tabla.

La campaña. Los días 15 y último de cada mes, entre las 12:00 y las 12:15, esperamos **el triple de usuarios simultáneos** que en hora punta, y que la mayoría inicie sesión en los **primeros 5 minutos**. Qolqe opera todos los días de 6:00 a 24:00 sin reinicios.

Operación	%
Consultar saldo y perfil	30 %
Consultar movimientos	15 %
Transferir a otro usuario Qolqe	15 %
Ver y buscar contactos	10 %
Recargar un celular	10 %
Consultar deudas y pagar servicios (6 % consulta, 4 % pago)	10 %
Otras (límites, alertas, solicitudes, dispositivos, sesiones)	10 %

Lo que esperamos del servicio:

- Consultas e inicio de sesión: el **95 %** debe responder en menos de **800 ms**.
- Operaciones de dinero (transferencias, recargas y pagos): el **95 %** debe responder en menos de **1.5 segundos**.
- En hora punta, **no más del 1 %** de las operaciones puede fallar, y la plataforma debe sostener el volumen de la hora punta sin acumular retrasos.
- Durante el pico de la campaña se acepta una **degradación razonable** del servicio, pero **ninguna operación de dinero puede quedar a medias ni aplicarse dos veces**.

Lo que necesitamos saber: si cumplimos en hora punta y en el pico de la campaña, y **hasta cuántos usuarios simultáneos** podemos crecer antes de dejar de cumplir.

5. Recomendaciones

Datos de prueba

- Las reglas de negocio aplican también bajo carga (límite diario, máximo de recargas al día, bloqueo tras 3 intentos fallidos): si varios VUs comparten usuario, las fallas serán de datos y no de rendimiento.
- Preparen los datos con los endpoints de soporte: `POST /api/pruebas/reiniciar` y `PUT /api/pruebas/saldo`.
- Un `422` por regla de negocio (por ejemplo, `LIMITE_DIARIO_EXCEDIDO`) indica un problema en los **datos**, no en el rendimiento. Sepárenlos en el análisis.
- Para verificar que ninguna operación de dinero quedó a medias o duplicada, comparen saldos y movimientos antes y después de la prueba.

Entorno

- Si trabajan en **local**, la herramienta y la aplicación compiten por el mismo CPU. Cierren otras aplicaciones y vigilen también el CPU de la herramienta: **si supera ~80 %, el resultado no es válido** (están midiendo a la herramienta, no a Qolqe).
- Con **dos equipos**, la red de la universidad puede bloquear la conexión entre laptops; usen el punto de acceso de un celular. Qolqe debe escuchar en `0.0.0.0` y el firewall debe permitir el puerto.
- En **Codespaces**, la latencia de internet se suma al tiempo de respuesta: compárenlo con una ejecución de 1 VU como referencia.
- En cualquier opción, los resultados son una **línea base relativa**, no la capacidad de producción. Documéntenlo en la sección 6 del informe.

Recursos (CPU y memoria)

La mayoría de herramientas **no** mide los recursos del servidor; mídanlos aparte, sobre el **proceso de Qolqe**. Registren la hora de inicio y fin de cada prueba para cruzar los recursos con los resultados.

Entorno	Cómo medir
Windows	Administrador de tareas o Monitor de recursos
macOS	Monitor de actividad
Linux	`top`, `htop` o `pidstat -p <PID> 1`
Docker	`docker stats`
Cualquiera	Script con `psutil` (al lado), que guarda un CSV para graficar

```
# pip install psutil · uso: python monitor.py <PID>
# (Ctrl+C para detener)
import csv, sys, time, psutil
p = psutil.Process(int(sys.argv[1]))
with open("recursos.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["hora", "cpu_%", "memoria_MB"])
    while True:
        w.writerow([time.strftime("%H:%M:%S"),
                    p.cpu_percent(interval=1),
                    round(p.memory_info().rss / 1e6, 1)])
        f.flush()
```

6. Entregables

Dónde	Qué
`tests/performance/`	Scripts, configuración de cada escenario y datos de prueba (o el script que los genera)
`tests/performance/resultados/`	Reportes de la herramienta, CSV de recursos y gráficos de cada ejecución
Informe del proyecto, sección 4	Informe de Pruebas de Rendimiento (plantilla adjunta)

Tag `parte-3` antes del viernes de la semana 14.

7. Exposición (15 min + 5 min de preguntas)

1. Herramienta elegida y comparación con otras herramientas.
2. Flujos críticos, umbrales y supuestos.
3. Modelo de carga y de dónde sale cada valor.
4. Demostración en vivo.
5. Resultados frente a los umbrales, cuello de botella y recomendaciones.

Ambos integrantes deben poder explicar cualquier decisión. Entre grupos hay de 5 a 10 minutos de transición para preparar el equipo.

8. Evaluación (20 puntos)

Criterio	Logrado	Pts
Investigación de la herramienta	Comparación argumentada con otras herramientas	3
Diseño de escenarios	Flujos críticos justificados, operaciones ponderadas, *think time* y datos variados por VU; cada valor trazado al requerimiento	5
Tipos de prueba	Carga y estrés ejecutadas; pico y resistencia ejecutadas o descartadas con justificación	3
Análisis frente a los umbrales	Umbrales y supuestos explícitos; p95, tasa de error y throughput comparados, con gráficos; cuello de botella con evidencia	5
Exposición y demo	Ejecución en vivo; ambos integrantes responden con solvencia	4